Rec_Exp-11

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pksj4>mongosh
Current Mongosh Log ID: 69ac75cf9cc587e1a87c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-02-26T12:53:32.501+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use Rec-11
switched to db Rec-11
Rec-11> db.students.insertMany([
| { name: "Prashant", age: 20, course: "AIML" },
| { name: "Mariam", age: 21, course: "AIML" },
| { name: "Aditya", age: 20, course: "AIML" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69ac761a9cc587e1a87c2907'),
    '1': ObjectId('69ac761a9cc587e1a87c2908'),
    '2': ObjectId('69ac761a9cc587e1a87c2909')
  }
}
Rec-11> db.students.find()
[
  {
    _id: ObjectId('69ac761a9cc587e1a87c2907'),
    name: 'Prashant',
    age: 20,
    course: 'AIML'
  },
  {
    _id: ObjectId('69ac761a9cc587e1a87c2908'),
    name: 'Mariam',
    age: 21,
```

```
    course: 'AIML'
  },
  {
    _id: ObjectId('69ac761a9cc587e1a87c2909'),
    name: 'Aditya',
    age: 20,
    course: 'AIML'
  }
]
Rec-11> db.students.updateOne(
| { name: "Prashant" },
| { $set: { age: 19 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Rec-11> db.students.deleteOne({ name: "Aditya" })
{ acknowledged: true, deletedCount: 1 }
Rec-11>
```

Welcome | server.js

server.js > ...

```javascript
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/practiceDB")
.then(()=> console.log("MongoDB Connected"))
.catch(err => console.log(err));

app.listen(3000,()=>{
console.log("Server running on port 3000");
});
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS node

```
PS C:\Users\pksj4\OneDrive\Documents\College\WP\exp-11> node server.js
Server running on port 3000
MongoDB Connected
```

```javascript
const userSchema = new mongoose.Schema({
name:String,
email:String,
age:Number
});

const User = mongoose.model("User",userSchema);

app.get("/add", async(req,res)=>{
const user = new User({
name:"Prashant",
email:"prashant@gmail.com",
age:20
});

await user.save();
res.send("User Added");
});
```

localhost:3000/add

User Added

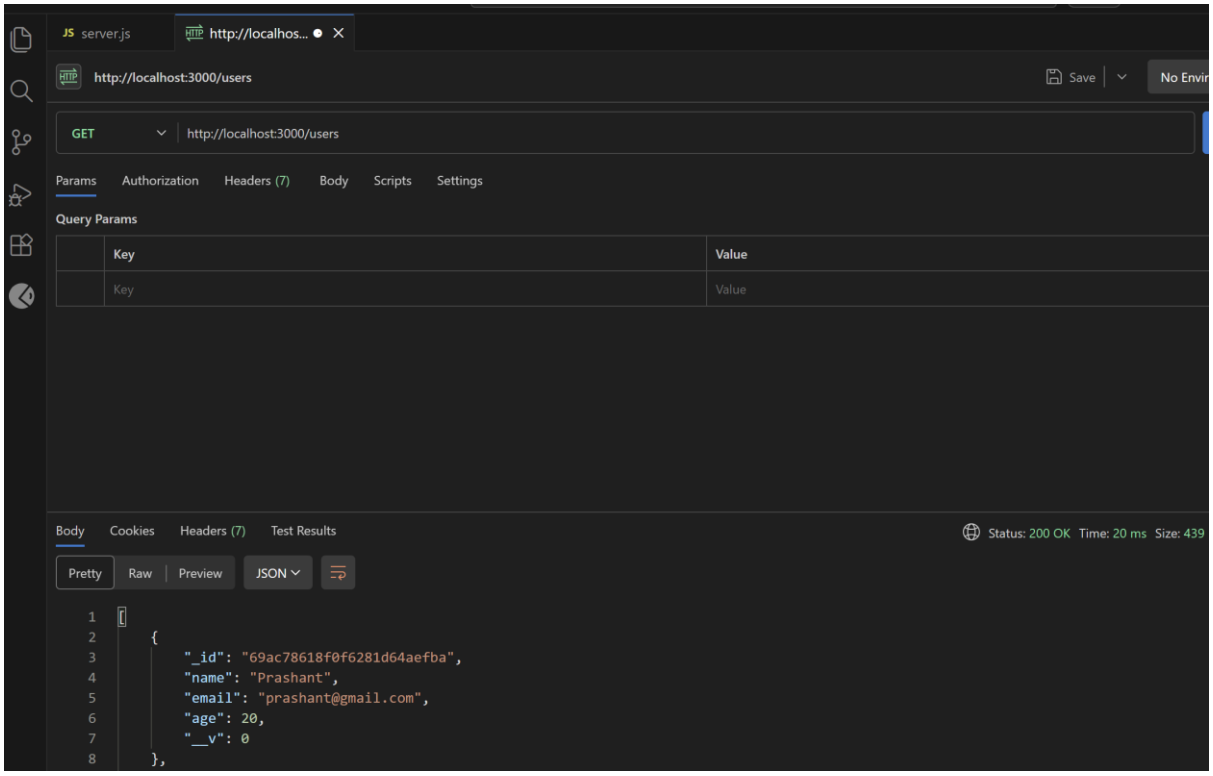
```
app.get("/users", async(req,res)=>{
const users = await User.find();
res.json(users);
});
```

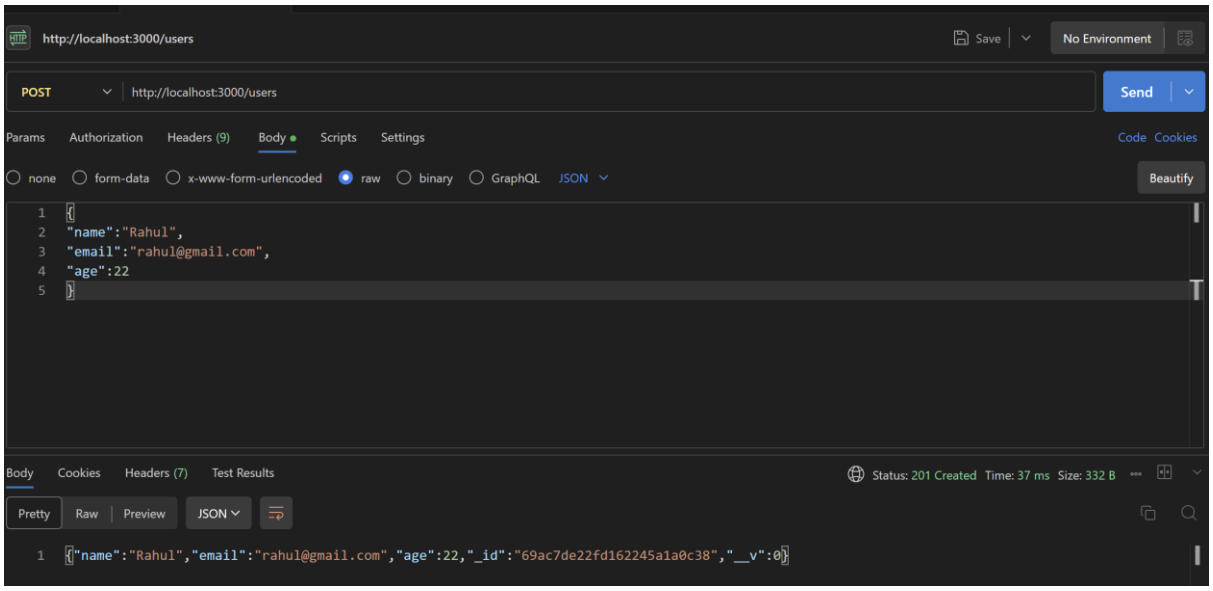
localhost:3000/users

Pretty print


```
[
  {
    "_id": "69ac78618f0f6281d64aefba",
    "name": "Prashant",
    "email": "prashant@gmail.com",
    "age": 20,
    "__v": 0
  }
]
```

```
app.post("/users", async(req,res)=>{
const user = new User(req.body);
await user.save();
res.status(201).json(user);
});

app.get("/users", async(req,res)=>{
const users = await User.find();
res.json(users);
});

app.put("/users/:id", async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    res.json(user);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.delete("/users/:id", async(req,res)=>{
await User.findByIdAndDelete(req.params.id);
res.send("User deleted");
});
```


server.js
http://localhos...
http://localhost:3000/users
Save
No Envir
GET
http://localhost:3000/users
Params
Authorization
Headers (7)
Body
Scripts
Settings
Query Params
Key
Value
Key
Value
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 20 ms
Size: 439
Pretty
Raw
Preview
JSON
[
{
"_id": "69ac78618f0f6281d64aefba",
"name": "Prashant",
"email": "prashant@gmail.com",
"age": 20,
"__v": 0
},


http://localhost:3000/users
Save
No Environment
POST
http://localhost:3000/users
Send
Params
Authorization
Headers (9)
Body
Scripts
Settings
Code
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Beautify
{
"name":"Rahul",
"email":"rahul@gmail.com",
"age":22
}
Body
Cookies
Headers (7)
Test Results
Status: 201 Created
Time: 37 ms
Size: 332 B
Pretty
Raw
Preview
JSON
{"name":"Rahul","email":"rahul@gmail.com","age":22,"_id":"69ac7de22fd162245a1a0c38","__v":0}

http://localhost:3000/users/69ac7de22fd162245a1a0c38

Save | No Environment

PUT http://localhost:3000/users/69ac7de22fd162245a1a0c38 Send

Params Authorization Headers (9) Body Scripts Settings Code Cookie

none form-data x-www-form-urlencoded raw binary GraphQL J... Beautif

```
1 {
2 "age":25
3 }
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 28 ms Size: 327 B

Pretty Raw Preview JSON

```
1 {"_id":"69ac7de22fd162245a1a0c38","name":"Rahul","email":"rahul@gmail.com",
    "age":25,"__v":0}
```

DELETE http://localhost:3000/users/69ac7de22fd162245a1a0c38 Send

Params Authorization Headers (9) Body Scripts Settings Code Cookie

none form-data x-www-form-urlencoded raw binary GraphQL J... Beautify

```
1 {
2 "age":25
3 }
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 26 ms Size: 239 B

Pretty Raw Preview HTML

```
1 User deleted
```

```
const userSchema = new mongoose.Schema({
name:{
type:String,
required:true
},
email:{
type:String,
required:true
},
age:{
type:Number,
min:18
}
});

app.post("/users", async(req,res)=>{
try{
const user = new User(req.body);
await user.save();
res.status(201).json(user);
}
catch(err){
res.status(400).json({error:err.message});
}
});
```

POST http://localhost:3000/users Send

Params Authorization Headers (9) Body Scripts Settings Code Cookies

none form-data x-www-form-urlencoded raw binary GraphQL J Beautify

```
1 {
2 "name":"Rahul"
3 }
```

Body Status: 400 Bad Request Time: 4 ms Size: 312 B

Pretty Raw Preview JSON

```
1 {"error":"User validation failed: email: Path `email` is required."}
```

http://localhost:3000/users Save No Environment

POST http://localhost:3000/users Send

Params Authorization Headers (9) Body Scripts Settings Code Cookies

none form-data x-www-form-urlencoded raw binary GraphQL J Beautify

```
1 {
2 "age":10
3 }
```

Body Status: 400 Bad Request Time: 4 ms Size: 408 B

Pretty Raw Preview JSON

```
1 {"error":"User validation failed: email: Path `email` is required.,
    name: Path `name` is required., age: Path `age` (10) is less than
    minimum allowed value (18)."}
```

http://localhost:3000/users/123

Save | No Environment

PUT http://localhost:3000/users/123 Send

Params Authorization Headers (9) Body Scripts Settings Code Cookies

none form-data x-www-form-urlencoded raw binary GraphQL J Beautify

```
1 {
2 "age":10
3 }
```

Body Status: 500 Internal Server Error Time: 7 ms Size: 357 B

Pretty Raw Preview JSON

```
1 {"error":"Cast to ObjectId failed for value \"123\" (type string) at
    path \"_id\" for model \"User\""}
```